

# Library Loan Tracker

A Full-Stack Book Circulation System with MongoDB Aggregation Analytics

|                         |                                                           |
|-------------------------|-----------------------------------------------------------|
| <b>Student</b>          | Aarya Patil                                               |
| <b>Course / Program</b> | MSc Data Engineering & Cloud Computing                    |
| <b>Date</b>             | September 12, 2026                                        |
| <b>Tech Stack</b>       | Node.js, Express.js, MongoDB (Mongoose), EJS, Bootstrap 5 |

## 1. Project Overview

For this exam project I built a small library circulation system called Library Loan Tracker. The idea came from wanting something with a clear one-to-many relationship between two collections (books and loans) that I could still finish inside a short time window, while leaving enough room to write a few real MongoDB aggregation pipelines rather than just basic CRUD screens. The application lets a librarian register books, issue a loan against a book, mark a loan as returned, and view three aggregated reports computed directly from the database.

The system is a classic three-tier web app: Express handles routing and request logic on the backend, MongoDB (through Mongoose) stores the data, and EJS templates styled with Bootstrap 5 render the pages on the server before sending them to the browser. I kept the whole thing to two collections — Book and Loan — since a loan simply references the book it belongs to and records who borrowed it and when.

### 1.1 Data Model

- Book: title, author, category, and totalCopies (how many physical copies the library owns).
- Loan: a reference to the borrowed book, the member's name, a borrowDate, and a returnDate that stays null until the book comes back.

Because returnDate is null for anything still checked out, most of the interesting logic in the app — active loans, overdue loans, availability counts — comes down to querying and aggregating on that one field.

### 1.2 Application Pages

- Dashboard — total titles in the catalog, how many loans are currently active, and how many of those are overdue.
- Book Inventory — the full catalog with category, total copies, and copies currently available.
- Loan Tracking — a form to issue a new loan plus a table of loans, filterable between active, returned, and all records.
- Analytics — three MongoDB aggregation pipelines rendered as report tables.

## 2. Screenshots — Core Screens

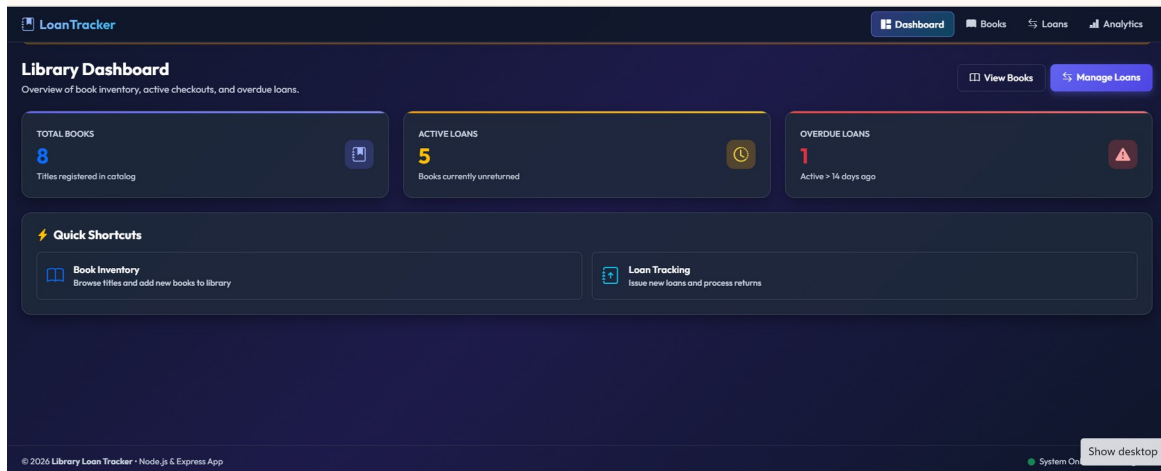


Figure 1. Dashboard showing total books, active loans, and overdue loans.

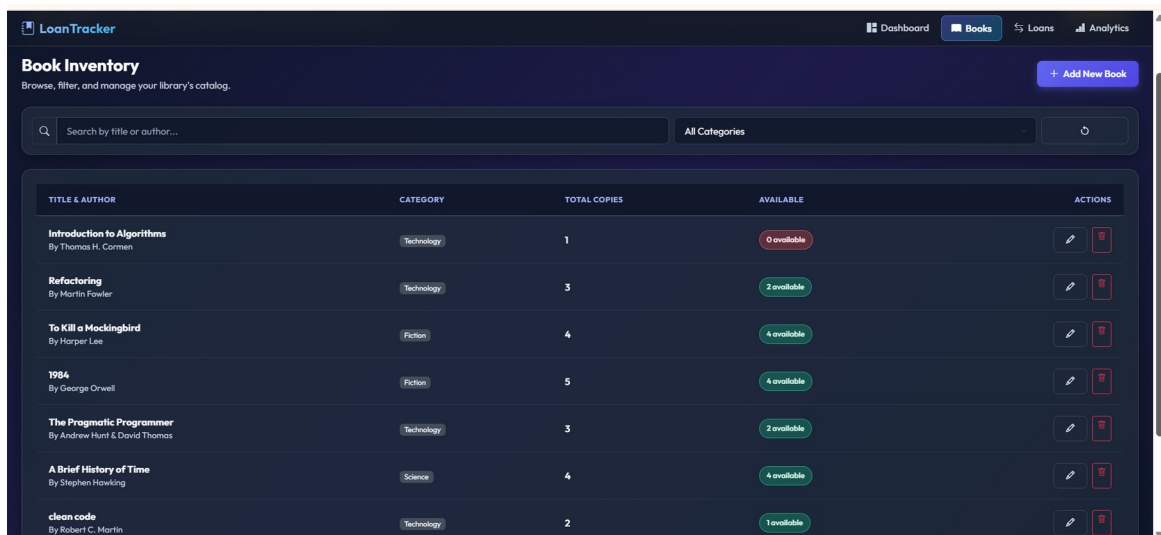


Figure 2. Book Inventory — catalog view with live availability per title.

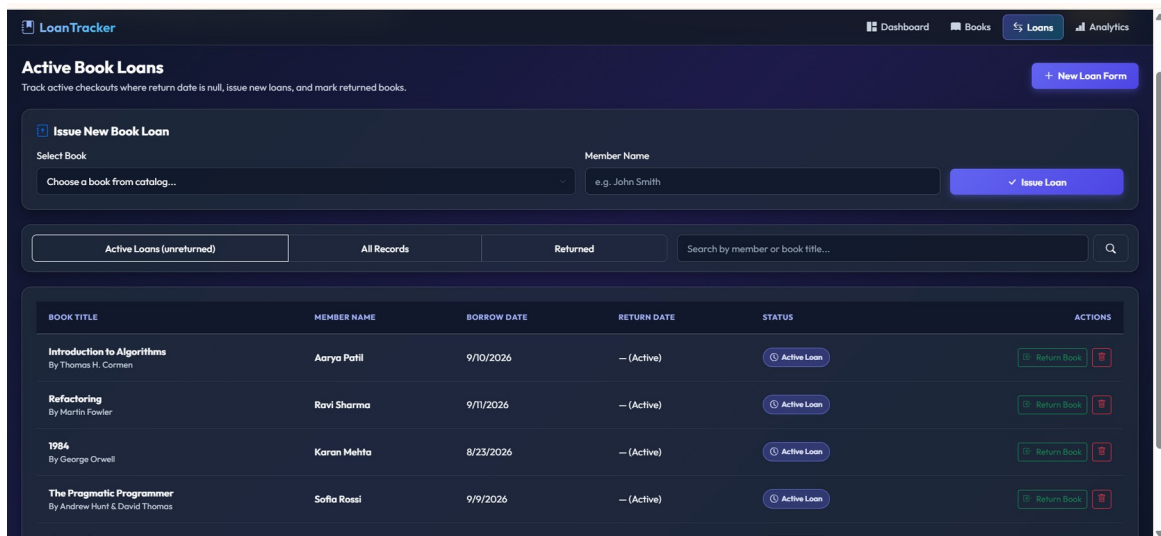


Figure 3. Loan Tracking — issuing a new loan and viewing active (unreturned) loans.

### 3. Analytics: Aggregation Pipelines

The part of this project I spent the most time on was the analytics page, since it's the one place where the database is doing real work instead of a simple `find()`. All three reports below are computed with Mongoose's `aggregate()` and rendered straight into Bootstrap tables — nothing is calculated in JavaScript afterwards.

#### 3.1 Top 5 Most-Borrowed Books

Groups all loan records by book and counts how many times each title has been checked out, then sorts descending and keeps the top five.

```
db.loans.aggregate([
  { $group: { _id: "$bookId", timesBorrowed: { $sum: 1 } } },
  { $sort: { timesBorrowed: -1 } },
  { $limit: 5 },
  { $lookup: { from: "books", localField: "_id",
    foreignField: "_id", as: "book" } },
  { $unwind: "$book" },
  { $project: { title: "$book.title", author: "$book.author",
    category: "$book.category", timesBorrowed: 1 } }
]);
```

#### 3.2 Currently Overdue Loans (> 14 Days)

Matches active loans (`returnDate` is still null) whose `borrowDate` is older than 14 days, which is the loan period I set for the library.

```
const cutoff = new Date(Date.now() - 14 * 24 * 60 * 60 * 1000);
db.loans.aggregate([
  { $match: { returnDate: null, borrowDate: { $lt: cutoff } } },
  { $lookup: { from: "books", localField: "bookId",
    foreignField: "_id", as: "book" } },
  { $unwind: "$book" },
  { $project: { "book.title": 1, memberName: 1, borrowDate: 1 } }
]);
```

#### 3.3 Books Never Borrowed

Left-joins every book against the loans collection and keeps only the books that came back with an empty match — i.e. titles nobody has ever checked out.

```
db.books.aggregate([
  { $lookup: { from: "loans", localField: "_id",
    foreignField: "bookId", as: "loanHistory" } },
  { $match: { loanHistory: { $size: 0 } } },
  { $project: { title: 1, author: 1, category: 1, totalCopies: 1 } }
]);
```

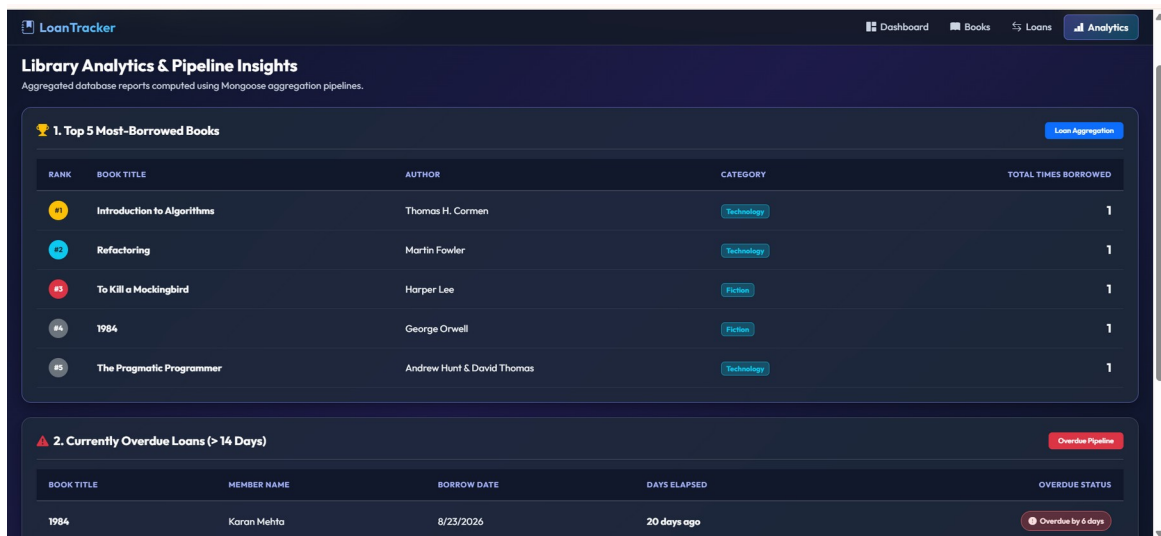


Figure 4. Analytics page — most-borrowed books and overdue loans.

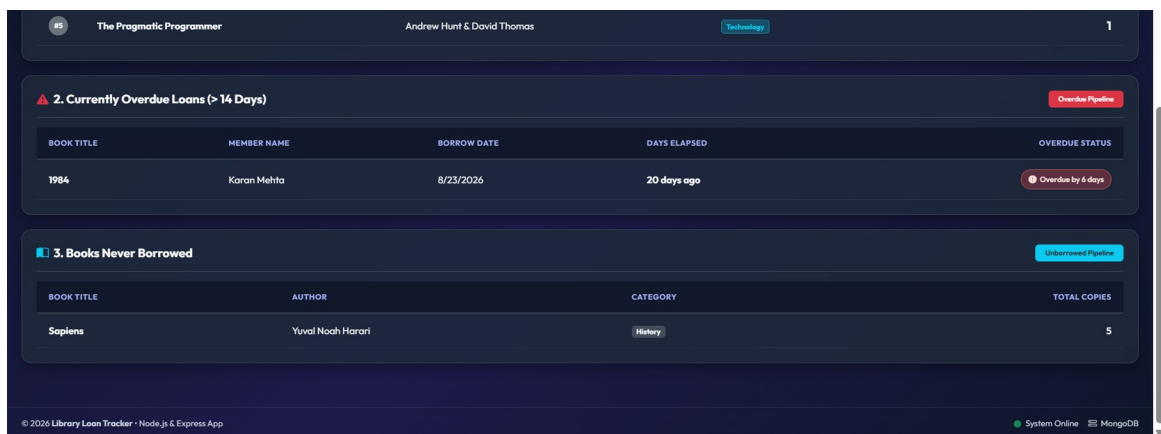


Figure 5. Analytics page (continued) — overdue detail and books never borrowed.

## 4. Verification & Reflection

I tested the system end to end by adding eight books across four categories, issuing five loans, returning two of them, and manually backdating one loan's borrowDate by 20 days to confirm the overdue pipeline actually flags it (Karan Mehta's loan of 1984 in Figure 4 shows as overdue by 6 days, which matches the 20-day borrow date against the 14-day cutoff). The dashboard counters, the loan table filters, and all three analytics tables update correctly and match what I'd expect from the raw data.

The main thing I'd improve with more time is moving the 14-day overdue threshold and the loan period into a config value rather than hardcoding it, and adding pagination to the book catalog since it will not scale well once the collection grows past a page or two. Overall, the project met its goal of demonstrating a working data model plus non-trivial aggregation logic within a short build window.

*System status: fully functional, tested locally on Node.js with a MongoDB backend.*